RecallFabric

Implementation Plan — Team Reference

Core idea: RecallFabric is a standalone AI memory service. It sits outside the AI model and provides persistent, relevant, structured, and portable memory to AI applications.

Key principle: The AI model generates the answer. RecallFabric manages the memory.

1. What Are We Building?

RecallFabric is a backend memory service/API. An AI application communicates with RecallFabric, which stores and retrieves user memories and supplies relevant context to the AI model.

```
User → AI Application → RecallFabric → Memory Database → RecallFabric → AI Model → Response
```

2. Problem We Are Solving

AI systems need useful long-term context. RecallFabric should preserve useful information and retrieve it later when it is relevant. It should also update changed information and track where each memory came from.

3. Overall Architecture

The first architecture consists of an AI application, the RecallFabric service, PostgreSQL with pgvector, and an embedding model.

```
AI Application
↓
RecallFabric
• Memory API
• Memory Engine
• Retrieval Engine
• Memory Manager
↓
PostgreSQL + pgvector
↑
Embedding Model
```

4. Phase 1 — Define the Memory Format

A memory is more than a text string. It also needs identity, ownership, type, confidence, importance, source, timestamps, and status.

```
{
"id": "mem_001",
"user_id": "user_123",
"content": "User is learning Spring Boot",
"type": "skill",
"importance": 0.86,
"confidence": 0.94,
"source": "conversation_102",
"status": "active"
}
```

5. Phase 2 — Build the Database

Use PostgreSQL + pgvector. PostgreSQL handles normal database operations, while pgvector enables semantic/vector search so memories can be found by meaning rather than only exact keywords.

```
Memory content + metadata + embeddings + timestamps + versions + provenance
```

6. Phase 3 — Build the Memory API

Build the backend service with Python and FastAPI. Expose basic CRUD and retrieval operations.

```
POST /memories
GET /memories
GET /memories/{id}
PUT /memories/{id}
DELETE /memories/{id}
POST /memories/search
```

7. Phase 4 — Add Embeddings

Convert memory text into an embedding using an embedding model and store the vector alongside the memory. Do the same for user queries.

```
Memory text → Embedding model → Vector → PostgreSQL/pgvector
```

8. Phase 5 — Build Memory Retrieval

Convert the user's query into an embedding, perform semantic search, obtain candidate memories, rank them, and return the most relevant ones.

```
Query → Embedding → Vector Search → Candidate Memories → Ranking → Relevant Memories
```

9. Phase 6 — Add Memory Ranking

Vector similarity alone is not enough. Rank memories using semantic relevance, importance, confidence, recency, and access frequency.

```
Final Score = semantic relevance + importance + confidence + recency + access frequency
```

10. Phase 7 — Connect an AI Model

Connect the first LLM. RecallFabric retrieves relevant memories, constructs additional context, and sends that context with the user's request to the LLM. The LLM generates the answer.

```
User question
↓
RecallFabric retrieves memories
↓
Context is constructed
↓
LLM API
↓
AI response
```

11. Phase 8 — Build Memory Extraction

After conversations, identify information that should become persistent memory. A memory-extraction component can classify candidate memories and assign importance and confidence before storage.

```
Conversation → Candidate memory → Evaluate → Importance/Confidence → Embedding → Store
```

12. Phase 9 — Prevent Useless Memories

The system must distinguish useful long-term information from temporary conversation. Useful memories can include preferences, goals, skills, and stable user information. Casual statements should normally not become memories.

13. Phase 10 — Handle Duplicate Memories

If a new memory is highly similar to an existing memory, avoid creating unnecessary duplicates. Consolidate or update the existing memory instead.

```
New memory → Similarity check → Duplicate? → Yes → Update/consolidate existing memory
```

14. Phase 11 — Handle Changing Information

When information changes, preserve history while marking the older memory as superseded and the new memory as active. This prevents outdated information from being treated as current.

```
Old memory → superseded
New memory → active
```

15. Phase 12 — Add Provenance

Every memory should record where it came from and why it was created. This makes the system explainable and easier to debug and evaluate.

```
Memory → source conversation + creation reason + confidence + timestamps
```

16. Phase 13 — Add Memory Management

Build a dashboard where users/developers can view, search, edit, disable, and delete memories. The dashboard supports transparency, debugging, demonstrations, and evaluation.

17. Phase 14 — Make It Model-Independent

Create provider adapters so RecallFabric is not tied to one LLM. The same memory layer should work with OpenAI, Claude, Gemini, and eventually local models.

```
RecallFabric → OpenAI / Claude / Gemini / Local LLM
```

18. Phase 15 — Connect External AI Applications

Expose RecallFabric as infrastructure that multiple AI applications can use. Different applications and models can use the same portable memory store, subject to authentication and permissions.

19. Phase 16 — Add MCP/API Integration

In addition to REST, consider MCP or other standard tool interfaces so compatible AI agents can call operations such as search_memory and store_memory.

20. Phase 17 — Security and Privacy

Implement authentication, authorization, encryption, user isolation, deletion controls, and audit logging. A user must never be able to access another user's memories.

21. Phase 18 — Evaluation

Compare three setups: an LLM without memory, a basic vector-memory system, and RecallFabric. Measure whether RecallFabric improves retrieval and answer quality while keeping latency and memory overhead reasonable.

```
Measure: retrieval precision, retrieval recall, answer quality, hallucination rate, latency, memory
efficiency, conflict handling, and cross-model consistency.
```

Final Architecture

The completed system separates memory management from language generation.

```
AI Application
↓
RecallFabric
• Memory API
• Memory Engine
• Retrieval Engine
• Ranking Engine
• Memory Manager
• Provider Layer
↓
PostgreSQL + pgvector

Provider Layer → OpenAI / Claude / Gemini / Local LLM
```

Recommended Development Order

Milestone	What we build
M1	Project setup
M2	PostgreSQL + pgvector
M3	Memory schema
M4	Basic Memory API
M5	Embedding generation
M6	Semantic memory retrieval
M7	Memory ranking
M8	Connect first LLM
M9	Automatic memory extraction
M10	Deduplication / consolidation
M11	Memory updates / conflicts
M12	Provenance
M13	Dashboard
M14	LLM provider adapters
M15	MCP / external integrations
M16	Security / privacy
M17	Evaluation framework
M18	Benchmarking + research results

Team Takeaway

Do not think of RecallFabric as an AI that remembers. Think of it as memory infrastructure that gives AI models persistent, relevant, structured, explainable, and portable context.

The AI remains responsible for understanding and generating language. RecallFabric is responsible for remembering, storing, retrieving, ranking, updating, and managing context.